Session_12 activities worksheet

Online handouts: Pendulum power spectrum graphs, listings of ode_test_class.cpp and classes, multifit_test.cpp and multimin_test.cpp.

In this Session, we'll take a look at nonlinear least-squares fitting with GSL.

Your goals for this week:

- Take a quick look at some pendulum power spectra.
- Try out and extend the adaptive ode code with classes.
- Try out nonlinear minimization on a demonstration problem.
- Try out nonlinear least-squares fitting on a demonstration problem.

Please work in pairs (more or less) and ask the instructors as you go.

A) Pendulum Power Spectra [Session_11 Follow-up]

Look at the handout with the three rows of plots showing the time dependence of a pendulum on the left and the corresponding power spectrum on the right plotted in terms of frequency = 1/period.

1. *How can you most accurately determine the single frequency in the first row from the graph on the left?*

   → You can approximate the period of the oscillations T from the peaks and troughs. Then you can determine the frequency f from 1/T or the angular frequency omega from 2pi/T.

2. Identify (and write down) two of the frequencies in the second row *from the plot on the left*. Do they agree with the plot on the right?

   →Using crude approximations for the period, we can find one oscillation period is T=26s and the other is T=10s (the latter found by using the fast frequency between two peaks that may not be equal). This gives us frequencies of about 0.04Hz and 0.1Hz, which roughly agree with the plot on the right.

3. *What is the characteristic of the power spectrum in the third row that is consistent with a chaotic signal?*
   →The spectrum is very “flattened” or spread out over many frequencies, and we cannot see real sharp, distinct peaks, which implies chaotic motion.

B) Multidimensional Minimization with GSL Routines

The basic problem is to find a minimum of the scalar function f(xvec) [scalar means that the result of evaluating the function is just a number] where xvec = (x_0, x_1, ... , x_N) is an N-dimensional vector. This is a much harder problem for N > 1 than for N=1. The GSL library offers several algorithms; the best one to use depends on the problem to be solved. Therefore, it is useful to be able to switch between different choices to compare results. GSL makes this easy. Here we'll try a sample problem. For more details, see the online GSL manual under "Multidimensional Minimization" (linked on Carmen).

The routines used here find *local* minima only, and only one at a time. They have no way of determining whether a minima is a global one or not (we'll return to this point later).

1. Look at the test code "multimin_test.cpp" and compare it to the online GSL documentation under "Multidimensional Minimization" (there is also a handout copy). Identify the steps in the minimization process. *What classes might you introduce for this code?*
   → In this code, we define a target function f(x) and df(x) and initialize parameters and the starting point. We allocate a GSL minimizer and set the solver state. We then start an interation loop to move the current point toward the minimum and then test its convergence. Then we cleanup and deallocate the GSL workspace. As for classes, I can think of two classes you could have to encapsulate this process: an "ObjectiveFunction" class to hold the parameters and to evaluate vectors, and a "Minimizer" class that contains the "engine" that solves and iterates, or sets the algorithm we want to use.

2. Compile and link the program with "make_multimin_test" and run it. Your results may differ from what is in the comments of the code. *Adjust the program so that your answer is good to 10^{-6} accuracy. What is it that is found to that accuracy? (E.g., is it the positions of the minimum or something else?)*
   → The accuracy is from the tolerance variable. This is the accuracy for the magnitude of the gradient, or the slope of the function, checking when it's nearly 0. So with 1e-6 tolerance, we are checking when the derivative is less than 1e-6.

3. Modify the function minimized so it is a function of three variables (e.g., x, y, z), namely a three-dimensional paraboloid with your choice of minimum. (Be sure to change ALL of the relevant functions.) *Is the minimum still found?*
   →Yes, the minimum is still found.

4. *What algorithm is used initially to do the minimization? Modify the code to try steepest_descent and then one of the other algorithms. Which is "best" for this problem?*
   →The original algorithm is the Fletcher-Reeves conjugate gradient algorithm from conjugate_fr. I tried the steepest_descent and then I tried vector_bfgs2. For this problem, bfgs2 operates the best, finding the 3D minimum in only 3 steps, while fr takes longer, and steepest_descent doesn't find the minimum in 100 iterations.

C) Nonlinear Least-Squares Fitting with GSL Routines

Using the nonlinear least-squares fitting routines from GSL is similar to using the GSL minimizer (and involves a special case of minimization). Your main job is to figure out from the documentation (see Carmen) and this example from GSL (only slightly modified from what you will find in the documentation) how to use the routines. Here we briefly explore the sample problem, which also illustrates how to create a "noisy" (i.e., realistic) test case ("pseudo-data").

The model is that of a weighted exponential with a constant background:
$y = A\, e^{-\lambda t} + b$
You'll generate pseudo-data at a series of time steps t_i (in practice, t_i is taken to be 0, 1, 2, ...). Noise is added to each y_i in the form of a random number distributed like a gaussian with a given standard deviation $sigma_i$. We'll revisit the GSL functions that generate this random distribution in the next Session; for now just accept that it works.

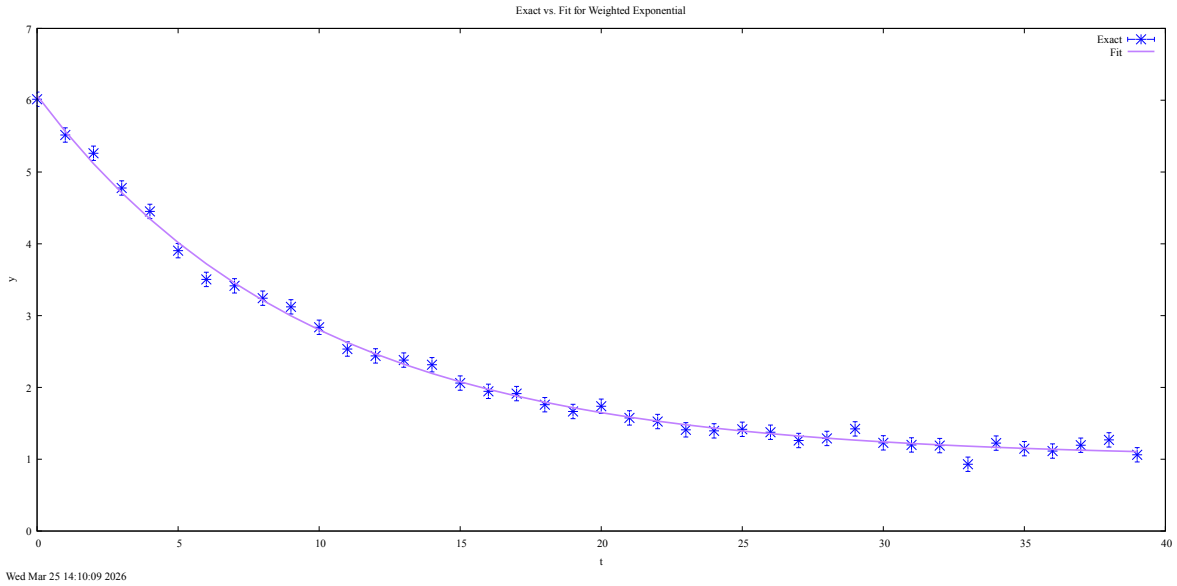
1. Take a look at the multifit_test.cpp code (there is a printout), run it (compile and link with make_multifit_test) and figure out the flow of the program. *What is the fitting ("objective") function to be minimized? What role does $sigma_i$ play?*

   →The function being minimized is the expb_f function, which returns the residuals for each point (like a chi-square). If that's the case, then sigma[i] represents the standard deviation, and its role involves weighting, determining how much influence a data point has on the final result.

2. *What is the "Jacobian" for? (Check the online documentation for the answer!)*
   → The Jacobian is like the multi-dimensional matrix version of a derivative. Each entry in the matrix is a partial derivative with respect to some parameter, which the solver uses to find the steps it should take and to calculate uncertainties.

3. *Modify the code so that you can plot both the initial data and the fit curve using gnuplot. Look up how to add error bars for the data in gnuplot. [Hint: Try "help set style", "help errors", and "help plot errorbars" for some clues.] Attach a plot.*

   →The attached plot is below:

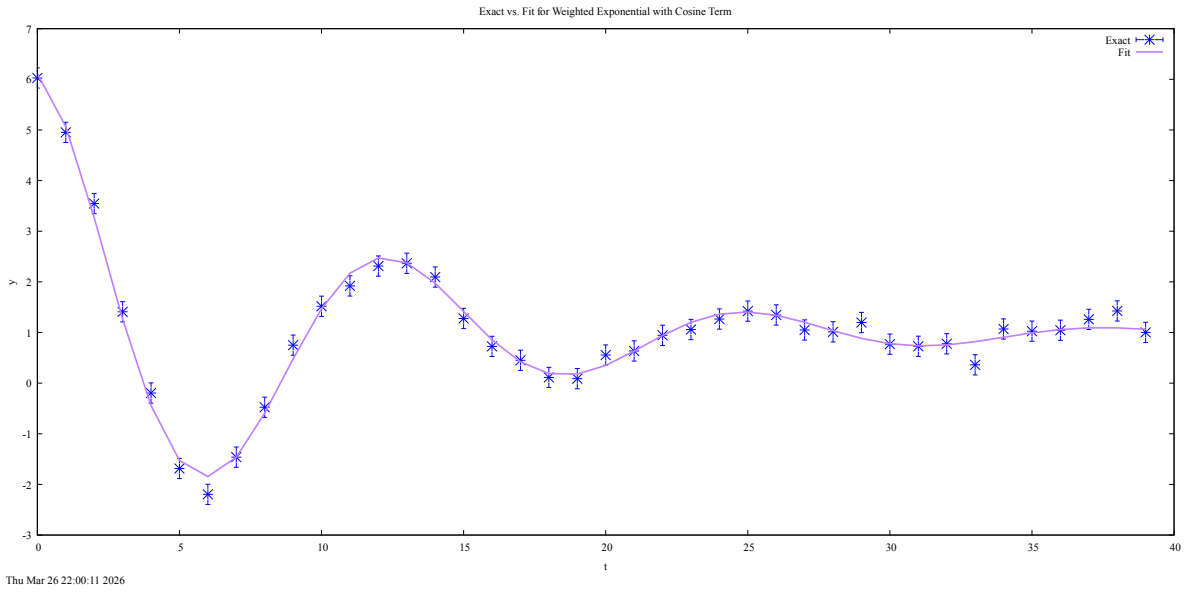


4. The covariance matrix of the best-fit parameters can be used to extract information about how good the fit is. *How is it used in the program to estimate the uncertainties in the fit parameters? How do these uncertainties scale with the magnitude of the noise? Do they scale with the number of time-steps used in the fit?* (I.e., if you change the "amplitude" of the gaussian noise, how do the errors in the fit parameters change?)

   →The program estimates the uncertainties using the Jacobian matrix using the inline err function, which takes the square root of the diagonal of the covariance_ptr to get a standard deviation. The uncertainties appear to scale proportionally to the noise (double the noise, double the error bars roughly). It does seem that increasing the number of time-steps decreases the error, but not by a significant amount.

5. [Bonus: come back to this.] Modify the function by adding a fourth parameter and try to fit (and plot!) again. For example, you could change "A" to "A cos(omega t)" with omega another parameter. Your choice!

   →Done with A*cos(omega t) with omega=0.5, and it fits well! Plot below:



D) GSL Adaptive ODE Solver Revisited

In Session_11, we took a quick look at ode_test.cpp, which implemented the GSL adaptive differential equation solver on the Van der Pol oscillator. Here we look at a rewrite that uses classes.

1. The details of ode_test_class.cpp and the Ode and Rhs classes are described in detail in the Session_12 notes. Look at the printout while you go through the notes. *What questions do you have?*

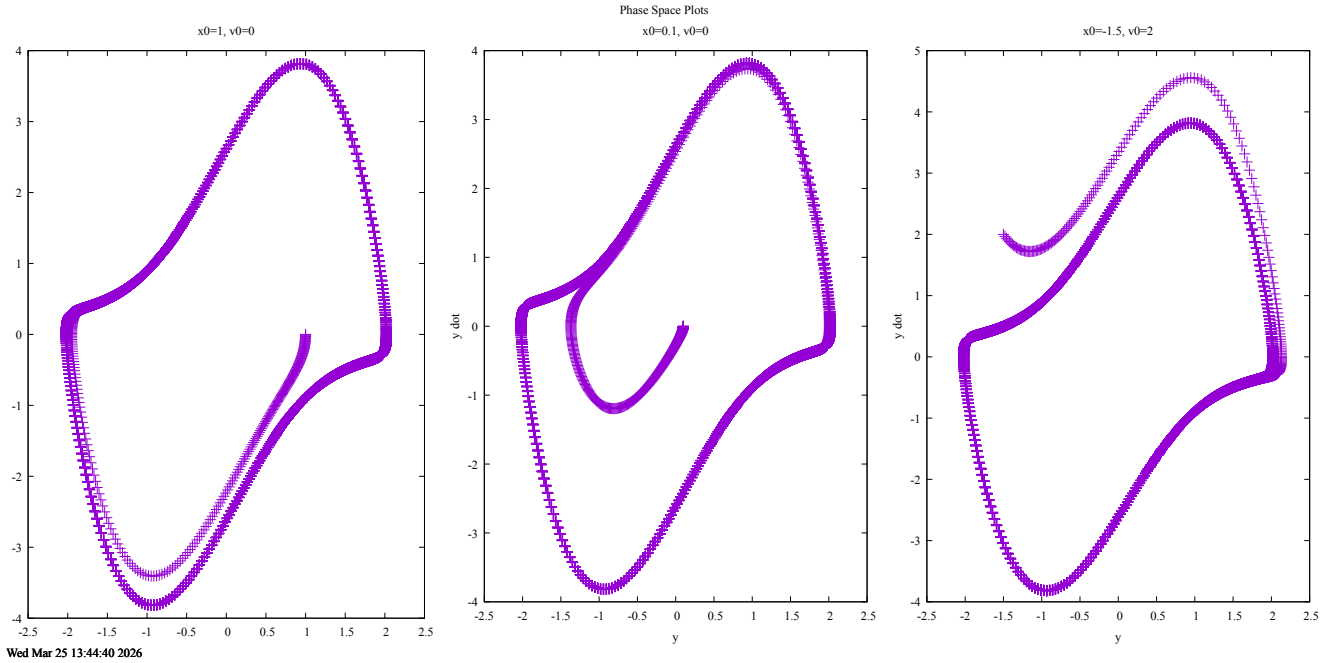
   →I don't have any questions currently (I do a lot of research with ODEs, so this looks familiar in many ways).

2. Add two additional calls to evolve_and_print so that all three initial conditions from Session_11, [x0=1.0, v0=0.0], [x0=0.1, v0=0.0], and [x0=-1.5, v0=2.0], are generated with the same run. *Did the three output files get generated? (You can try plotting to check correctness.)*

   → Yes, three output files were generated.

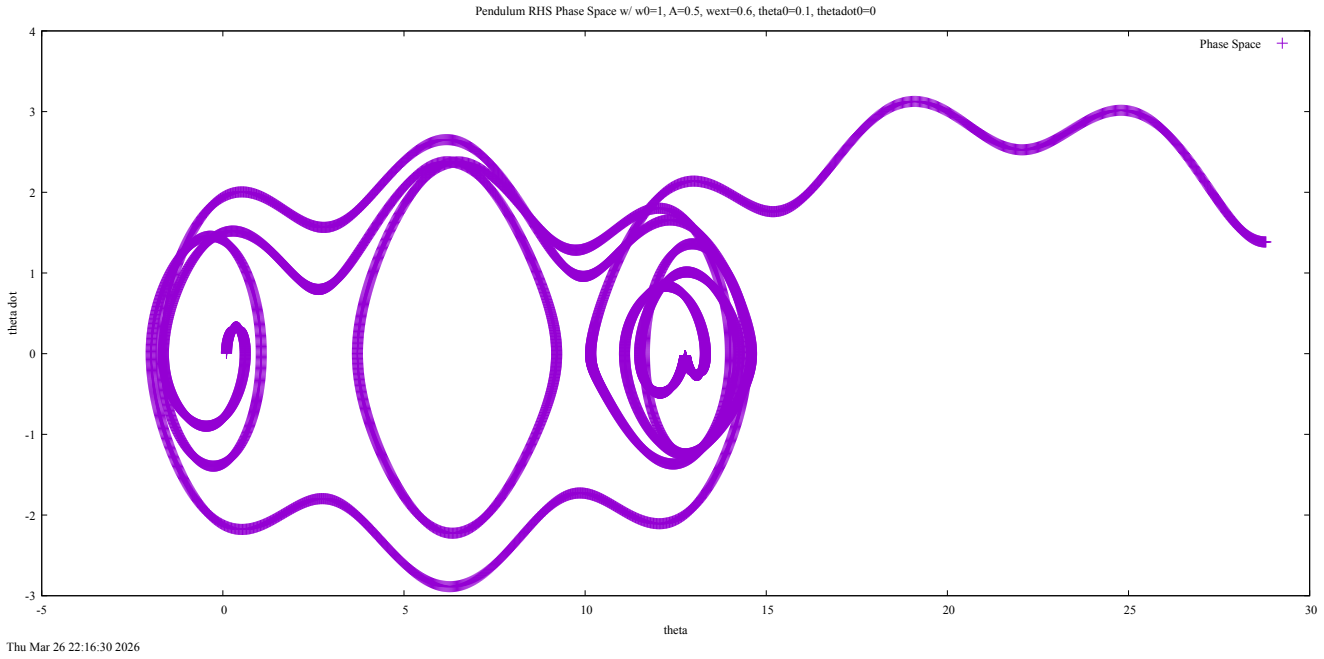
3. Add another instance of the Rhs_VdP class called vdp_rhs_2 with mu=3 and generate results for the same initial conditions. *Plot these. Is it still an isolated attractor?*

   → Yes, it does seem like there is still an isolated attractor with mu=3.



4. [Bonus. Come back to this if you finish everything else.] Add another class Rhs_Pendulum that implements the pendulum differential equation with natural frequency omega0 and a driving force with amplitude f_ext and frequency omega_ext. (You will want to modify or replace evolve_and_print.)

→ Done. Here is an example phase space I got:



E) Reflection

Write down which of the six learning goals on the syllabus this worksheet addressed for you and how:

→ This worksheet helped me learn more tools of computational physics with GSL fitting and minimization. I also understand physics through code, especially with the power spectrum and the relationships between period and frequency through it, and the power spectrum with chaos.